

lab03

Core Concepts in Generative AI

You'll learn: the mechanics of common generative model families (GANs, VAEs, diffusion), how latent spaces work, how to generate simple synthetic data, and how to evaluate generative quality with metrics like FID and BLEU.

Prereqs: Python, Google Colab (GPU optional for speed, CPU still okay).

Deliverables: screenshots, plots, and short bullet reflections for each lab.

Lab 1 — Latent Space Exploration (20–25 mins)

Goal: Understand how random noise vectors map to synthetic samples.

Setup

In []:

```
# This type of code is common in generative models (such as
autoencoders or GANs),
# where the "latent space" represents a compressed encoding of
data.
# Here, that latent space is being simulated with random points.

# # NumPy (np): Used to manipulate numeric arrays and generate
random numbers.
# Matplotlib (plt): Used for plotting.
import numpy as np
import matplotlib.pyplot as plt

# 2D latent space: random vectors
# `np.random.randn(500, 2)` generates a 500 × 2 array.
# Each row represents a point in a two-dimensional (2D) space.
# The values are generated from a standard normal distribution
# (mean 0, standard deviation 1).
# In other words, 500 random (x, y) points are being created,
# normally distributed around the origin (0, 0).
z = np.random.randn(500, 2)

# z[:,0]: all x-coordinates (first column).
```

```

# z[:,1]: all y-coordinates (second column).
# plt.scatter(...) draws a scatter plot.
# alpha=0.5 makes the points semi-transparent so they are easier
# to see when they overlap.
plt.scatter(z[:, 0], z[:, 1], alpha=0.5)

# Add the title "Random Latent Space Points".
plt.title("Random Latent Space Points")

# plt.show() displays the figure.
# El resultado es una nube de puntos centrada alrededor del origen,
# con forma circular (debido a la distribución normal en 2D).
plt.show()

```

Steps

1. Generate 2D latent points.
2. Map them to fake "images" (simple shapes). Example: circle radius proportional to $z[0]$, color shade proportional to $z[1]$.

In []:

```

# A new figure (fig) and axis (ax) are created for each iteration.
fig, ax = plt.subplots(figsize=(5, 5))

# Iterate 9 times (from i = 0 to i = 8).
for i in range(9):
    # Assume that z is a list (or array) of 9 elements,
    # where each element v contains at least two values (for
    example, v = [x, y]).
    v = z[i]

    # The size of the circle depends on the absolute value of the
    first component of v.
    radius = abs(v[0]) * 0.1

    # A color is created in RGB format.
    # The red channel depends on the absolute value of the second
    component of v.
    # The other two channels are fixed (0.5).
    color = (abs(v[1]) / 3, 0.5, 0.5)

    # Calculate position (x, y) on the 3x3 grid
    row = i // 3 # Row
    col = i % 3 # Column
    x = 0.2 + col * 0.3 # Horizontal separation
    y = 0.8 - row * 0.3 # Vertical separation

    # Center of the circle at (0.5, 0.5) (the center of the axis).
    # Radius scaled by radius / 50.
    circle = plt.Circle((x, y), radius, color=color)

```

```

ax.add_patch(circle) # The circle is added.

ax.set_xlim(0, 1) # The visible range of the chart is set.
ax.set_ylim(0, 1)
ax.set_aspect("equal") # Maintain proportions
ax.axis("off") # The axes are disabled for a clean view.

plt.show() # Display each figure in a separate window.

```

Checkpoint: 3–4 screenshots of your generated shapes.

Reflection: How small changes in z alter the outputs.

Lab 2 — Train a Tiny GAN on MNIST (30–40 mins)

Goal: Experience adversarial training (generator vs discriminator).

Setup

In []:

```

#!pip install -q torch torchvision
import torch, torch.nn as nn
from torchvision import datasets, transforms

```

Steps (simplified skeleton)

1. Load MNIST digits (torchvision.datasets.MNIST).
2. Define Generator: input noise $\rightarrow$ FC layers $\rightarrow$ 28×28 image.
3. Define Discriminator: input image $\rightarrow$ FC layers $\rightarrow$ probability real/fake.
4. One training loop for ~200 iterations (not full training, just to see process).

In []:

```

# DataLoader: used to load data in batches and shuffle them.
from torch.utils.data import DataLoader

# random: here it is used only to simulate random losses,
# since we have not yet trained a real network.
import random

# --- Dataset and DataLoader ---
# This defines how the images are processed before use:
# ToTensor(): converts each image (28×28 pixels in MNIST) to a
PyTorch tensor
# of type float with values between 0 and 1.
# Normalize((0.5,), (0.5,)): adjusts these values to be between -1
and 1
# → useful for neural network models (better numerical stability).

```

```

transform = transforms.Compose(
    [transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))]
)

# Download the MNIST dataset (handwritten digits, 60,000 images).
# root='data': Saves the files to a local folder called data/.
# train=True: Loads the training set (not the test set).
# transform=transform: Applies the transformations defined above.
dataset = datasets.MNIST(root="data", train=True, download=True,
transform=transform)

# The DataLoader takes the dataset and prepares it for training:
# batch_size=64: groups the images into batches of 64.
# shuffle=True: shuffles the data at each epoch
# (important so the model doesn't learn the dataset's order).
# Now loader is an iterable object that returns: losses_g, losses_d
= [], []
# where:
# imgs → Shape tensor [64, 1, 28, 28]
# labels → Tensor with the real digits (0-9)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

# --- Simulated training ---
# In a GAN:
# `losses_g` would store the generator losses.
# `losses_d` would store the discriminator losses.
losses_g, losses_d = [], []

# The training is performed during 2 epochs (complete passes through
the data).
for epoch in range(2):
    # Get the batches of images (we ignore the tags with _).
    for imgs, _ in loader:
        # simulates a generator failure (random number between 0 and
1).
        loss_g = random.random()
        # simulates a loss of the discriminator.
        loss_d = random.random()

        # Save those losses in lists.
        losses_g.append(loss_g) # Take the first 5 items from the
generator loss list.
        losses_d.append(
            loss_d
        ) # Take the first 5 items from the discriminator's loss
list.

    # It cuts the internal loop to process only one batch per
epoch
    # (faster for testing the code).

```

```

        break # just one iteration to keep it short

# Prints the first stored loss values.
print("Sample losses:", losses_g[:5], losses_d[:5])

# Interpretation:
#   The generator has simulated losses of xxx and xxx in the first
#   two epochs
#   (we only process one batch per epoch).
#   The discriminator has simulated losses of xxx and xxx in the
#   same epochs.

```

5. Visualize generated digits after a few steps.

Checkpoint: Loss curves & sample images.

Reflection: Why does adversarial training sometimes oscillate?

Lab 3 — Variational Autoencoder (VAE) Demo (25–30 mins)¶

Goal: See how VAEs learn a compressed latent representation and reconstruct inputs.

Setup

In []:

```

from torch import optim

# Use torchvision FashionMNIST for variety

```

Steps

1. Build encoder: input → latent mean & logvar.
2. Reparameterize trick: $z = \mu + \sigma * \epsilon$.
3. Build decoder: latent → reconstructed image.
4. Train briefly on a small subset.
5. Compare original vs reconstructed samples side-by-side.

Checkpoint: side-by-side plots of originals vs reconstructions.

Reflection: Why do VAEs produce blurrier outputs than GANs?

Lab 4 — Diffusion Model Intuition (20–25 mins)¶

Goal: Simulate "forward" noise addition and "reverse" denoising steps.

Steps

1. Start with a grayscale MNIST digit.
2. Iteratively add Gaussian noise 10 times (forward process).

In []:

```
import numpy as np
import matplotlib.pyplot as plt

# `imgs` is a batch of MNIST images, in the form `[batch_size, 1, 28, 28]`.
# `imgs[0][0]` → selects the first image from the first channel
# (MNIST only has one channel, grayscale).
# `.numpy()` → converts the PyTorch tensor to a NumPy array,
# which is easier to manipulate for visualization and mathematical
operations.
img = imgs[0][0].numpy()

# Create a copy of the original image.
# This ensures that we don't directly modify the image.
# The image we'll iteratively add noise to is called "noisy".
noisy = img.copy()

# It is run 10 times, each time adding a little noise to the image.
for i in range(10):
    # generates a random noise array with normal distribution
    # (mean 0, standard deviation 1) and the same shape as the
    image (28,28).
    # 0.05 * ... → scales the noise to make it small (approximately
    5%).
    # noisy += ... → adds the noise to the current image.
    noisy += 0.05 * np.random.randn(*img.shape)

    # displays the image in grayscale.
    plt.imshow(noisy, cmap="gray")

    # displays the step number (0 to 9).
    plt.title(f"Step {i}")

    # render the image.
    plt.show()
```

3. Try a crude denoising (Gaussian filter).
4. Observe reconstruction quality.

Checkpoint: progression of noise addition + denoised output.

Reflection: How diffusion models gradually "learn to reverse noise."

Lab 5 — Evaluation Metrics (15–20 mins)¶

Goal: Try simple quantitative checks of generative quality.

Steps

1. Generate 100 fake digits with your GAN.
2. Compare with 100 real digits.
3. Compute a toy FID-like metric: mean & covariance difference of features.

In []:

```
from sklearn.metrics import pairwise_distances

# We simulated 10 real images and 10 "fake" ones
real = torch.randn(10, 28, 28).numpy()
fake = torch.randn(10, 28, 28).numpy()

# We calculate the average distance between pairs.
# Flatten each image into a vector. Example: (28,28) → (784,).
# Calculate the Euclidean distance between each pair of images
(fake[i], real[j]).
# Returns the average distance between all combinations.
dist = pairwise_distances(
    fake.reshape(len(fake), -1), real.reshape(len(real), -1)
).mean()

# That means that, on average, each generated (fake) image is at a
distance
# of xxx units (in pixel space) from the real images.
# This value is not an official GAN metric, like FID or IS
# (Frechet Inception Distance or Inception Score).
# It only measures differences in pixel space, not in visual or
semantic features.
# That's why it's called "toy distance": a simple and pedagogical
metric,
# useful for getting a quick, but not scientific, understanding.
print("Toy distance:", dist)
```

4. Try BLEU score with text.

Checkpoint: printed metric value + 2–3 sentences of interpretation.

Reflection: Why are evaluation metrics tricky in generative AI?

Grading Rubric (10 pts)

- Latent exploration + shapes (2 pts)
- GAN sample run & generated images (3 pts)
- VAE reconstructions (2 pts)
- Diffusion noise/denoise demo (2 pts)
- Evaluation metric reflection (1 pt)